Trainer notes: LaunchKit session opener

Oct 15, 2026 · @Riya

Goal and timing

The opener sets the end goal of the upskill and earns the agent harness. In 20 minutes, trainees learn what they'll build and the exact problem it solves, and see from their own runs what one LLM call can't do, what a LangGraph pipeline adds around it, and what that costs. Hands-on is copy-paste: trainees paste the provided TODO snippets and run the scoreboard. Everything that doesn't fit in 20 minutes is either a trainer demo or in the hands-on sheets' *After the session* sections.

By the end, trainees can:

- State the problem statement and the four requests that define done
- Explain what one LLM call does well (the creative part, rules that are written down) and what it can't do (memory, photos, sizes, rules nobody wrote down)
- Explain what a LangGraph pipeline adds (state, a checkpointer, a router, a review loop) and where it stops
- List the requirements that lead to an agent harness

Minutes	Segment	Slides	Format
0–2	LaunchKit, Harbor and its brand pack	3–4	Talk
2–3	Problem statement and definition of done	5–6	Talk
3–4	One LLM call: how it works, live	7–8	Demo
4–7	Hands-on: paste two TODOs, run the scoreboard	9	Hands-on
7–8	Scoreboard: one LLM call, and the festival brief	10	Debrief
8–9	A LangGraph pipeline: how it works, live	11	Demo
9–16	Hands-on: paste four TODOs, run the scoreboard	12	Hands-on
16–19	Scoreboard, the review loop, and requirements	13–14	Debrief
19–20	Bridge to the agent harness	15	Talk

The hands-on sheets' *After the session* sections (the model call up close, the browser UI, the reliability discussion, stretch tasks) are for afterwards.

Before the session

Every trainee needs

- Python 3.12 or newer, and `uv`
- git, and access to the KeyValue repo
- A coding agent they already use: Cursor, Claude Code, or another
- A key for KeyValue's LiteLLM proxy: `LITELLM_API_KEY` and `LITELLM_BASE_URL` in `.env`

Ask trainees to run `uv sync` and `cp .env.example .env`, add their key, run `uv run python -m launchkit.check_setup` until it says **Setup complete**, then add `LAUNCHKIT_IMAGES=placeholder` to `.env` and check out `session-01-llm-call`. A real image takes about a minute through the proxy and the scoreboard makes four, which a 3-minute hands-on can't absorb. Keep real images on your own machine for the demos.

Trainer checklist

☐ On `session-01-langgraph`, with a real key: `scoreboard --approach llm` gives 1/4; after pasting the four LangGraph TODOs, `scoreboard --approach graph` gives 4/4, in under a minute with placeholders
☐ `check_setup` passes, including the image check, so the key reaches `gpt-4.1-mini` and `gpt-image-1` through the proxy
☐ Asked the proxy admin about intermittent 401 errors (`Incorrect API key provided: sk-svcac…`). In dry runs up to 1 in 4 requests failed this way; it's the proxy's upstream key, not the trainee's
☐ Rehearsed the three live runs below on your own machine: the starter on `session-01-llm-call`, the festival brief after the LLM-call TODOs, and the festival brief on `session-01-langgraph` after TODO 4
☐ Pairs arranged: one drives, one navigates
☐ Slides open in full screen (press F)

Branches

Each branch contains the previous branch's solution, so a trainee who falls behind checks out the next branch and carries on.

Branch	Trainees do	Starts with
`session-01-00-setup`	Run `check_setup`	Shared code only
`session-01-llm-call`	Two TODOs in `approach_llm/app.py`	Shared code only
`session-01-langgraph`	Four TODOs in `approach_graph/graph.py`	LLM call solved
`session-01-harness`	Next segment	LangGraph solved

Three ways to run things

Command	Use it for
`uv run python -m launchkit.ask R1`	One request. Prints every model call (what was sent, what came back), the poster, Harbor's rules checked and the images. Add `--approach graph` on the LangGraph branch, and `--no-image` to skip the paid image call.
`uv run python -m launchkit.ui`	The same in a browser, at `http://localhost:8000`. Harbor's four requests are one click away, and it remembers each conversation, so R1 then R2 works as a follow-up. Pick *LangGraph* under *Approach* on the LangGraph branch.
`uv run python -m launchkit.scoreboard --approach llm`	All four requests in order, each marked PASS or FAIL with a reason. Use `--approach graph` on the LangGraph branch.

The repo runs on the real model. Results vary from run to run, which is itself a teaching point; `complete()` deliberately skips the proxy's reply cache so that they do. A real image takes about a minute through the proxy.

Kickoff (3 minutes)

LaunchKit, Harbor and its brand pack (slides 3–4, 2 min)

- Say: LaunchKit is a marketing assistant platform. A brand describes a concept and gets finished posters back. By the end of the upskill, each pair builds its Poster Ad agent, combining five skills.
- Slide 4: Harbor's launch is around forty poster variants. Rules, words, sizes and iteration are what make it hard.
- Harbor's rules are concrete and checkable by code: word limits, banned claims, placement sizes. Point out what the brand pack can't list: every claim Harbor has never made, like "survives drops". It comes back in both debriefs.

Problem statement and definition of done (slides 5–6, 1 min)

- Read the problem statement aloud. Walk R1–R4: a concept becomes a poster, a follow-up edit, a photo, a resize of that poster. The scoreboard decides, not opinions.
- Show of hands: how many of the four can one LLM call pass?

One LLM call (5 minutes)

How it works, live (slides 7–8, 1 min)

1. Slide 7: a prompt with Harbor's rules, one call, parse the JSON, then the image model gets the picture description only.
2. Slide 8: run the starter: `uv run python -m launchkit.ask R1 --no-image` on `session-01-llm-call`. The model writes a decent poster idea, but as prose, so code gets `no JSON object in the model's reply`; in the dry run it also invented a tagline, "Stay Fresh. Stay Tidal." That's what TODO 1 fixes.

> Don't promise the room broken word limits or banned words. Once the rules are in the prompt, `gpt-4.1-mini` followed every written rule in all 51 dry-run posters. The failure worth showing is the rule nobody wrote down.

Hands-on (slide 9, 3 min)

On `session-01-llm-call`, trainees paste the two snippets from the hands-on sheet (trainer copy: `hands_on_llm.md`) into `approach_llm/app.py` and run `uv run python -m launchkit.scoreboard --approach llm`. Paste them on your own machine too, for the debrief.

TODO	Solves
1. Rules in the prompt	The model gets Harbor's rules and the JSON shape
2. Every placement size	Plain code makes all five placements; the model can't

Expected: R1 passes; R2, R3 and R4 fail.

Scoreboard and the festival brief (slide 10, 1 min)

- Compare with the show of hands. One call can't remember the conversation (R2), use a photo (R3) or resize "that poster" (R4).
- Run the festival brief live: `uv run python -m launchkit.ask --no-image "Summer festival poster: the Tidal bottle survives being dropped on concrete and keeps water ice cold all day."` The rules check says ✓, yet the copy claims something like "Ready for any drop" (4 of 5 dry runs). Harbor never said that, and bans "unbreakable". Ask: what would catch it?

A LangGraph pipeline (11 minutes)

How it works, live (slide 11, 1 min)

1. Slide 11: `route`, then `write` → `check` → `picture`, with a retry loop from `check` back to `write`, `give_up` after two tries, and a `resize` branch that uses no model. `write` is last session's prompt.

2. **State** is one dict every node reads and adds to; a **checkpointer** keeps it per conversation (`thread_id`). The graph decides what runs next, never the model.

Hands-on (slide 12, 7 min)

On `session-01-langgraph`, trainees paste the snippets from the hands-on sheet (trainer copy: `hands_on_langGraph.md`) into `approach_graph/graph.py`, running `uv run python -m launchkit.scoreboard --approach graph` after each of TODOs 1–3 (each run takes under a minute with placeholders), then the festival brief once after TODO 4. Paste them on your own machine too.

TODO	Solves	Scoreboard
Start	Same as the LLM call; a poster request makes just the picture now	1/4
1. Checkpointer (1a) and the last poster in the prompt (1b)	R2: follow-up edits	2/4
2. The photo from the state	R3: a concept given as a photo	3/4
3. Router node	R4: resize "that poster"; placement sizes now come only from this route, with no model	4/4
4. Review loop	The festival brief: a reviewer flags unapproved claims and the loop sends them back to `write`	4/4

If anyone asks why 1a alone doesn't fix R2: saved state does nothing until the prompt uses it. Pairs who run short skip the scoreboard after TODOs 1 and 2 and run it once after TODO 3.

Scoreboard, the review loop and requirements (slides 13–14, 3 min)

- Slide 13: 1/4 for the call; the graph also starts at 1/4 and reaches 4/4 once the TODOs are in.
- Run the festival brief live with the graph: `uv run python -m launchkit.ask --approach graph --no-image "Summer festival poster: the Tidal bottle survives being dropped on concrete and keeps water ice cold all day."`. Point at the reviewer's `=> UNSUPPORTED` line and the second `write` call carrying "Fix these problems". In the dry run: drop claims in 4 of 5 posters before TODO 4, 0 of 8 after. The loop doesn't trigger on every run; if it doesn't, run it again.
- Then the cost: R4 alone needed a router node, a resize node and a router prompt, and every new kind of request means more of the same. The reviewer is a model too: on a test set it let 3 of 12 bad claims through. What checks the checker?
- Slide 14: read the requirements out. The model chooses the next step, code enforces limits, actions are tools, state is shared, output is validated, and per-brand rules live with the brand.

Close and likely questions

Bridge (slide 15, 1 min). Say: "That set of requirements has a name: an agent harness. Next, we let the model choose its next step, inside limits our code enforces, on Agno."

Likely questions

- **"LangGraph can build agents too, can't it?"** Yes. The point is which responsibilities you still build yourself: per-brand instructions, skills, memory, limits and validation. The next segment names them.
- **"Wouldn't a better model just get it right?"** It reduces bad output, but no model can see a conversation it isn't given, use a photo it isn't sent, or resize a file it can't reach. As a reviewer, `gpt-4.1` scored 22 of 24 against `gpt-4.1-mini`'s 21.
- **"Why did my results differ from my neighbour's?"** The model isn't deterministic. That's one more reason code checks every reply.
- **"Why does the router ask two questions?"** One question kept misrouting requests. Let the model answer small questions; let code make the decision.
- **"Why did R2 make a whole new picture just to shorten the headline?"** Every edit goes through `picture`. Reusing the last picture for copy-only edits is the first stretch task.
- **"Why not let the image model draw the text?"** Image models misspell and can't follow brand fonts or safe zones, and words in pixels can't be resized to other placements. Code renders the text later (the supers skill).

Troubleshooting

Symptom	Fix
`AuthenticationError` with `Incorrect API key provided: sk-svcac…`	The proxy's own upstream key failed, not the trainee's. Run it again, and tell the proxy admin
Any other `AuthenticationError`, or no API key	Add `LITELLM_API_KEY` and `LITELLM_BASE_URL` to `.env` in the repo root, then run `check_setup`
"Can't load approach_llm" or "approach_graph"	Wrong branch: check out `session-01-llm-call` or `session-01-langgraph`
`No module named ...`	Run `uv sync`, and use `uv run` for every command
The score doesn't go up after a TODO	The snippet wasn't saved, or went in the wrong place; compare with the hands-on sheet
R2 still fails after TODO 1	Paste both parts: 1a (the checkpointer) and 1b (the last poster in the prompt)
The UI shows a gradient labelled PLACEHOLDER	"Skip the paid image call" is ticked, or `LAUNCHKIT_IMAGES=placeholder` is set in `.env`
A real image seems stuck	It takes about a minute through the proxy; use `--no-image` while iterating
In the UI, R2 has nothing to edit right after saving `graph.py`	Saving reloads the graph and resets its memory. Run R1 again, or start a new conversation
Two `ask` commands don't remember each other	Memory lives inside one running program. Use the UI or the scoreboard for conversations
The UI won't start: port 8000 is busy	Add `--port 8001`; the address is then `http://localhost:8001`
Changes to `launchkit/` don't show in the UI	Restart the UI. It reloads `app.py` and `graph.py` by itself, but not the shared code
Image model access error	Check the key's access to `gpt-image-1`; as a last resort set `LAUNCHKIT_IMAGES=placeholder` in `.env`
A pair falls behind	`git stash`, then check out the next branch
Rate limits	Stagger runs across pairs, or run the scoreboard once per pair